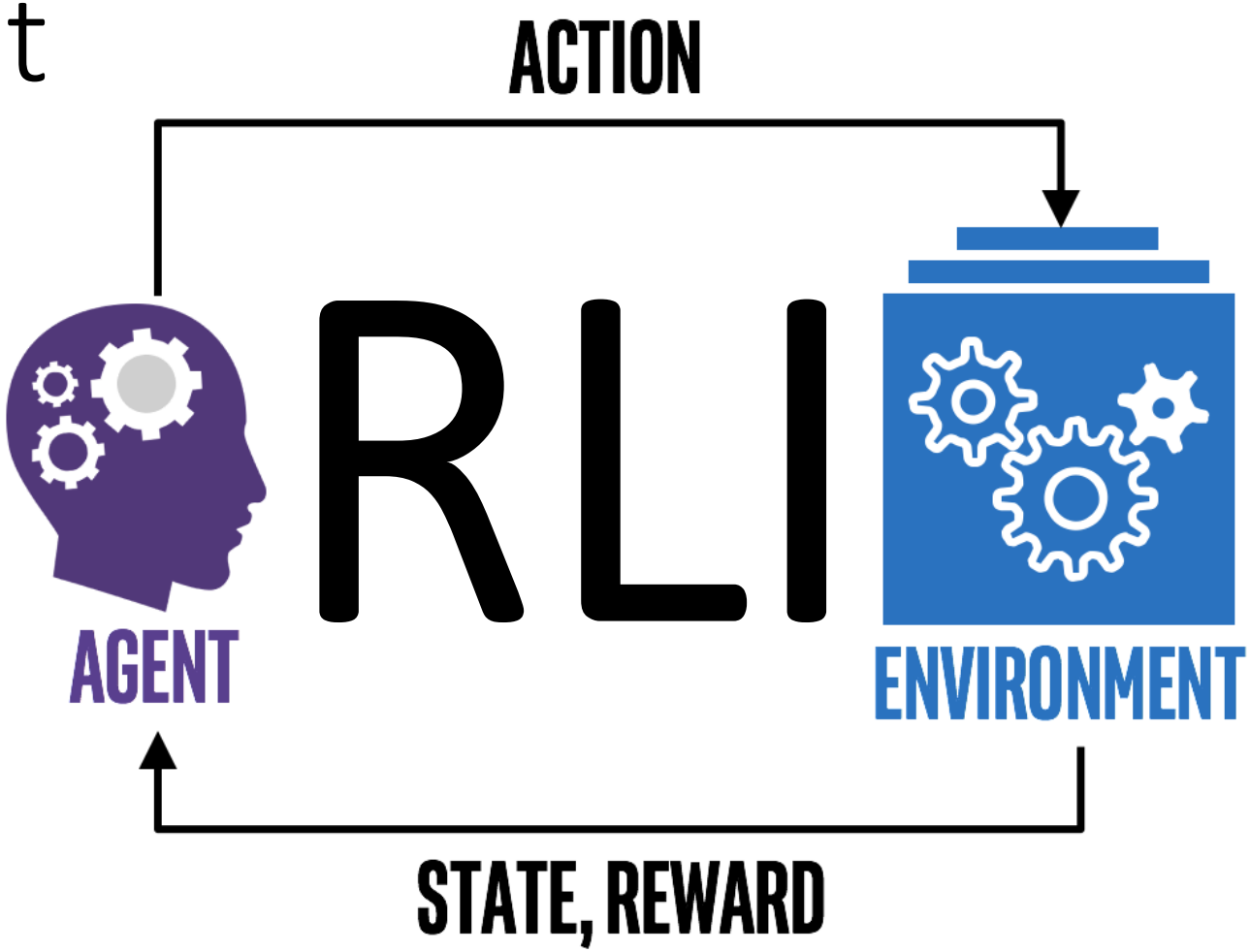
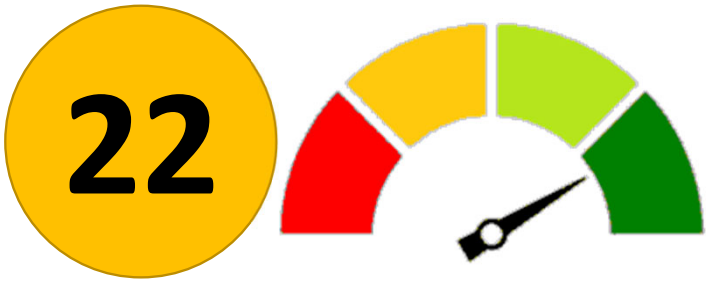
Reinforcement Learning Introduction





José Manuel Rey, IE

Reinforcement Learning Introduction Assignment 22.00

Group Assignment

Date: Mar/19 2026

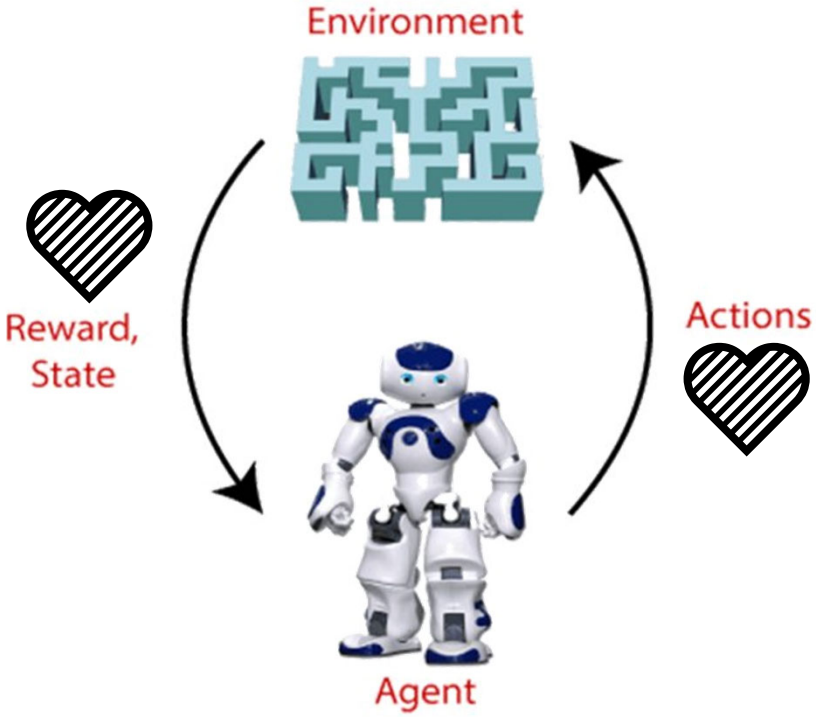
Due Date: Abr/29 2026 [SESSION 29]

Ongoing grading (maximum)

140 Group Assignment

Tinder© for RL

Let's find our best RL solution candidate for a given environment/problem



Understanding and building optimal RL solutions for specific types of problems/environments

Assignment Description Overview

Integrating conceptual/methodological/technical RL principles into practice

Assignment Part 01 → Analyze, Design, Build, Test/Evaluate, Compare and Explain/Justify a set of optimal solutions (policies) for different variations of the "Mountain-Car" Gymnasium environment *[70% points]*

- ***Note***: Additional description of the environment, examples of some of the deliverables and some initial example code for reference will we provided in subsequent sections of this document

Assignment Part 02 → Study, dissect, analyze and explain a paper/project/solution based on RL that in your opinion could be considered a best practice or instructive/remarkable example in this field *[30% points]*

- ***Note***: A tentative list of possible examples will provided orientatively in subsequent sections of this document

Assignment Description Overview – Part 01

- The objective of this part of the assignment is to create, develop, and analyze in depth, an approach for the optimal resolution of a given problem (“Mountain-Car”) under different conditions and constraints, and to conduct an analysis of both the effectiveness and optimization of the generated solutions, as well as their conceptual interpretation and interpretability.
- To this end, different types of representations of states and actions as well as various types of models for their solution must be analyzed, comparing the quality of the results generated, both in terms of mean values and statistical variability or fluctuation, as well as consistency.
- Regarding interpretability, comparative analyses of the generated policies must be conducted to assess their effectiveness from a conceptual (physical) perspective of the problem analyzed.

Assignment Description Overview – Part 02

- For the second part of the assignment, the student must conduct research and identify relevant examples and projects in the field of Reinforcement Learning (in any area of theoretical, research, prototypes, industrial of business problems), and perform an analysis and personal evaluation of their understanding of the proposed solution's suitability, the description of the methodology used in the project, the definition of states and actions, and the selection of the RL model and approach applied, as well as an analysis of the types of results obtained, and the methodology used for the analysis, interpretation, and explanation of those results.

Assignment Description Overview

For each part/question you should elaborate a self-contained directory-zip file with your ".py" and ".ipynb" files (derived from the documents delivered with the assignment), providing (when required) a clear concise explanation of the conceptual approach, the diagrams or models representing the problem, comments for the code used and the explanation of the solution

SUBMIT YOUR WORK BY ZIPPING TOGETHER THE FINAL CODE (COMPLETED WITH YOUR ANSWERS) IN A SINGLE SUBMISSION FILE NAMED:

"RLI_22_00 – Group {Group number XY}.zip"
(see the attached .xlsx for the groups)

- `RLI_BCSAI_C3{A|B}_2026 - Final Assignment Groups.xlsx`

Administrative and additional notes

- Organize yourselves in the assigned mini-groups:

  Only submit ONE zip file per group. Choose ONE member of each group for that submission. In case you send more than one, ensure that all the submissions by the different members are identical [as the system will choose just ONLY one of them (randomly) for the review and evaluation of the whole group] or state very clearly in the Blackboard submission which one should be considered as the valid one for reviewing

- As usual, **the code should run without warnings or errors** and in full stand-alone mode in a separate python virtual env. All additional required files should be included in the ZIP file, and if you were using any "novel" additional external library for your work, clearly indicate a brief description of its purpose, the version used and the "**`!pip install <package>`**" required for its installation

Key components/steps

- Design: state representations, rewards, types of RL algorithms, training strategies, hyperparameters …
  - Custom adaptation of standard gym environment when needed (wrappers or post-modification)
- Testbeds/Frameworks, Modularization (training/testing)
- Monitoring tools (Tensorboard)
- Evaluation
  - Analysis of (objective) performance vs (engineered) reward
  - Policy analysis:
    - numerical, mathematical/statistical, structural (functional)
    - Structural/topological analysis of the policies generated
  - Comparative Policy Analysis (among environment versions)
- Visualization tools
- Explanation tools (basic ML: regression, feature importance, …)

Evaluation Guidelines

As it can be observed from the Rubrics, the overall evaluation of the project would be:

- PROJECT CODE/DOCUMENT 70%
- PRESENTATION 30%

Use the following values as an orientation for the amount of work (and extension) expected in your final document.
At this stage we will be more concerned with the technical and methodological aspects of the project (specially those related to the RL nature of the project) than with the general formal aspects

Section	Extension Grading
Executive Summary Description	10
Value proposition (Design analysis)	20
Type of solution	15
Project Plan	35
Outcomes/Presentation of Results	10
Project deployment/execution (Testbed quality)	10

Use also the next attached slide (generic RL Roadmap) as a guideline for some of the points that should be adequately covered in your work

RL Roadmap [Generic Setup]

- Project Understanding and Methodological Roadmap
  - Problem understanding, definition and classification
    - Manageable number of features per state, number of states, number of actions
  - Methodological Roadmap
    - Approach and Strategy for solving the Problem
- States and Actions → Environment Structure
  - Methods and Processes implementation
  - The OpenAI Gym wrapper environment
- Rewards
- Agents
  - Learning agent
- Framework Selection

RL Roadmap [Iterative Loop]

- RL Method Selection
  - Models and Training
  - Training strategy
  - Methodological Roadmap
    - Approach and Strategy for solving (optimizing) the Problem
- Training
  - Training process
  - Training monitoring/tuning
- Performance evaluation
  - Hyper-parameter tuning
- Model Improvement
  - Opponent upgrade

Conclusions

- Analysis of results
  - Policy “interpretation” and analysis
  - Performance metrics summary
  - Conclusions
    - Lines of future development
- Abstract

Administrative Notes

- Instructions for GROUP ASSIGNMENT 22.00, which is due one day prior to SESSIONS 29 & 30
- Groups Assigned → based on your inputs for preferences [The composition of the Groups is informed in an attached document]
- For SESSIONS 29 & 30:
  - Each **group** will nominate a Coordinator (responsible for sending the assignment's documents or materials)[only the Coordinator is required to send the document on behalf of the other members of the Group]
  - 6 Presentations will be made → 15~20 minutes each
  - Presentations must be made collaboratively by all members of the team
  - 10~20 minutes for a joint discussion and exchange of impressions about the projects presented
  - 10~20 minutes for wrap-up and closing of the Course

SUBMIT YOUR WORK AS A STANDALONE .PDF OR .PPTX FILE OR BY ZIPPING TOGETHER IN SINGLE FILE THE ADDITIONAL FILES (PROJECT MAPS, REFERENCES, ETC) THAT YOU MAY DEEM NECESSARY

NAME YOUR SUBMISSION AS "RLI_22_00 – Group {Group number XY}.zip"

Notes for Grading

- **All members** of the group must participate collaboratively in the presentation in a freely organized/structured way
- Questions will be addressed by turn to every member of the group about questions related to the part or section he/she exposed or eventually about any other section or about general aspects regarding the concepts, theory, algorithms, technical or practical issues involved in the project presented (and related to the contents of the course) [an outline similar to that of a traditional oral exam]
- There will be an "*average*" grading for the Team as a whole, but optionally and only if circumstances require or advise it, **additional** points may be assigned to certain individuals within the team if disparities in contribution levels or knowledge regarding the project are detected. This will be probably an exceptional case but it is a prerogative of the evaluator.

Assignment Description
DELIVERABLES

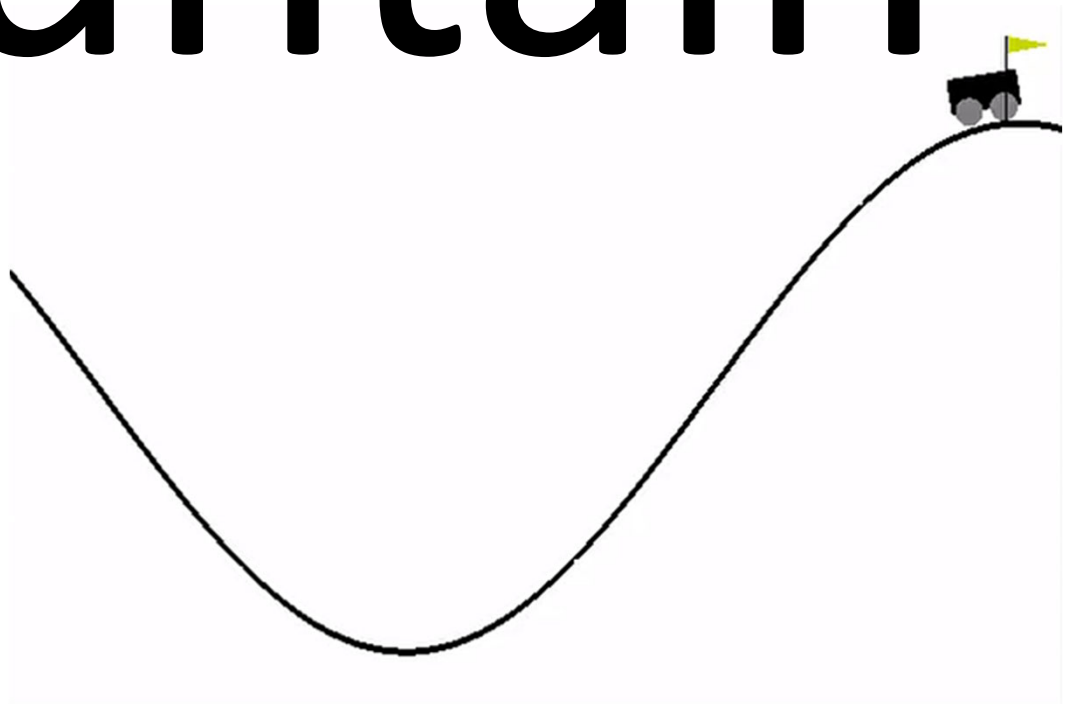
- (1) Code Framework/Testbed (Part 01)
  - Structured Jupyter Notebook, covering/describing all the required aspects of the solution, reproducible and stand-alone, producing all the models/information used or required for the presentation of results
- (2) Presentation Document (Part 01 + Part 02)
  - An executive summary of the solutions analyzed/built, but not a simple elevator pitch.
  - Adopt a paper-like approach: abstract/summary, elements considered for the analysis, relevant implementation details, visualization of outputs, comparison and interpretation of results, main conclusions
  - It must include an overview of the Conceptual/Technical components applied and the explanation of the main issues addressed (FOR INSTRUCTIONAL PURPOSES OF THE REST OF THE CLASS)
  - Content suited to the duration of the presentation
  - Note: for the “Part 02”, only this deliverable is required

Note: both deliverables must be posted/sent by the Coordinator before the Presentation day

Assignment
Background Information

The environment…

Mountain Car



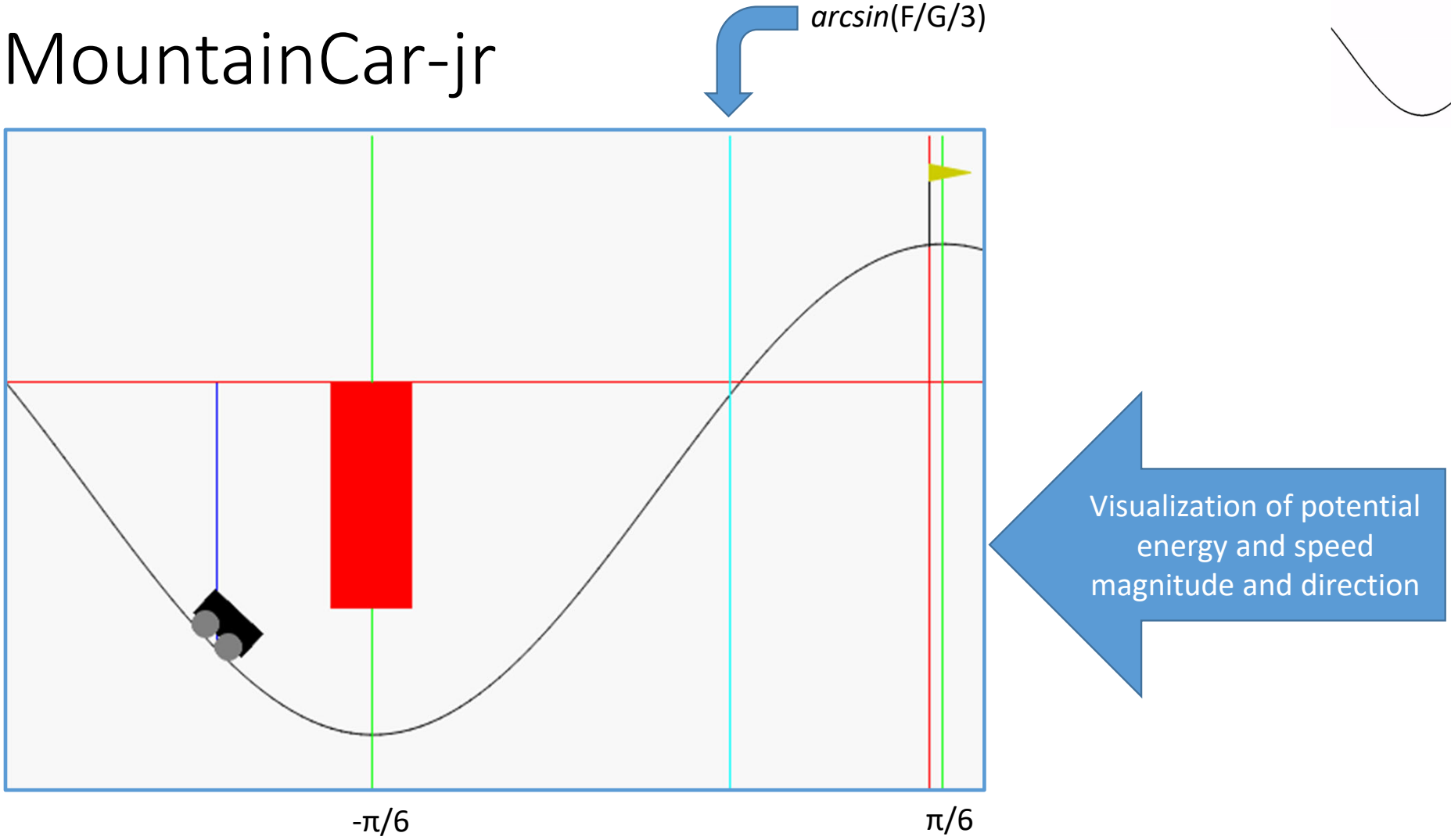
Mountain Car

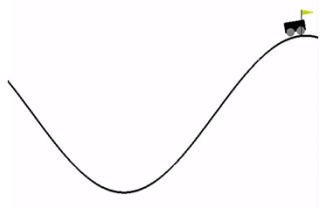
Mountain Car, a standard testing domain in Reinforcement learning, is a problem in which an under-powered car must drive up a steep hill. Since gravity is stronger than the car's engine, even at full throttle, the car cannot simply accelerate up the steep slope. The car is situated in a valley and must learn to leverage potential energy by driving up the opposite hill before the car is able to make it to the goal at the top of the rightmost hill.

The domain has been used as a test bed in various Reinforcement Learning papers.
The mountain car problem, although fairly simple, is commonly applied because:

- it requires a RL agent to learn on two continuous variables: position and velocity.
- For any given state (position and velocity) of the car, the agent is given the possibility of driving left, driving right, or not using the engine at all.
- In the standard version of the problem, the agent receives a negative reward at every time step when the goal is not reached; the agent has no information about the goal until an initial success.

MountainCar-jr



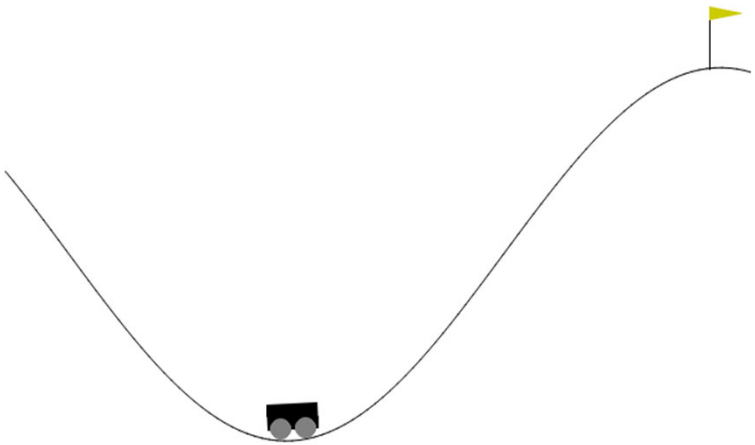


MountainCar
Gym Environment

The Mountain Car MDP is a deterministic MDP that consists of a car placed stochastically at the bottom of a sinusoidal valley, with the only possible actions being the accelerations that can be applied to the car in either direction.
The goal of the MDP is to strategically accelerate the car to reach the goal state on top of the right hill.
There are two versions of the mountain car domain in gymnasium:

- Discrete actions
- Continuous actions

This MDP first appeared in Andrew Moore's PhD Thesis (1990)



Mountain Car - Gymnasium Documentation
https://gymnasium.farama.org/environments/classic_control/mountain_car/

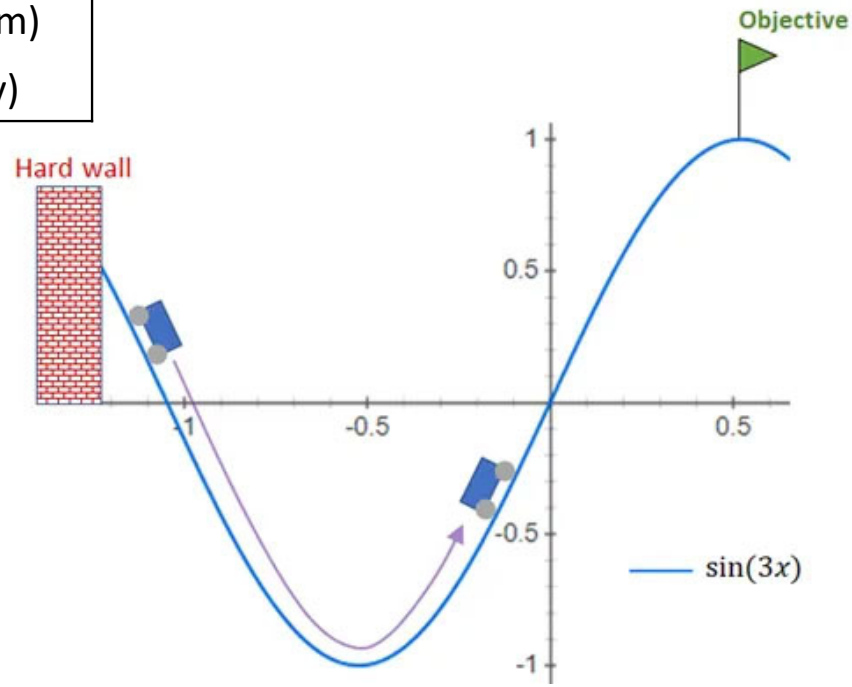
(*) This environment is part of the Classic Control environments which contains general information about the environment.

MountainCar Gym Environment

Observation Space

The observation is a ndarray with shape (2,) where the elements correspond to the following:

Num	Observation	Min	Max	Unit
0	position of the car along the x-axis	-1.2	0.6	position (m)
1	velocity of the car	-0.07	0.07	velocity (v)



MountainCar (Discrete actions) Gym Environment

Action Space	Discrete(3)
Observation Space	Box([-1.2 -0.07], [0.6 0.07], (2,), float32)
import	gymnasium.make("MountainCar-v0")

$$\frac{d}{dt}v(t) = (a(t) - 1)F - cos(3x(t))G$$

$$\frac{dx}{dt} = v(t)$$

Action Space

There are 3 discrete deterministic actions:

- 0: Accelerate to the left
- 1: Don't accelerate
- 2: Accelerate to the right

Transition Dynamics:

Given an action, the mountain car follows the following transition dynamics:

$velocity_{t+1} = velocity_t + (action - 1) * force - cos(3 * position_t) * gravity$

$position_{t+1} = position_t + velocity_{t+1}$

where force = 0.001 and gravity = 0.0025. The collisions at either end are inelastic with the velocity set to 0 upon collision with the wall.

The position is clipped to the range [-1.2, 0.6] and velocity is clipped to the range [-0.07, 0.07].

MountainCar (Continuous actions) Gym Environment

Action Space	Box(-1.0, 1.0, (1,), float32)
Observation Space	Box([-1.2 -0.07], [0.6 0.07], (2,), float32)
import	gymnasium.make("MountainCarContinuous-v0")

Action Space

The action is a ndarray with shape (1,), representing the directional force applied on the car.

The action is clipped in the range [-1,1] and multiplied by a power of 0.0015

Transition Dynamics:

Given an action, the mountain car follows the following transition dynamics:

$velocity_{t+1} = velocity_t + force * self.power - cos(3 * position_t) * gravity$

$position_{t+1} = position_t + velocity_{t+1}$

where force = 0.001 and.

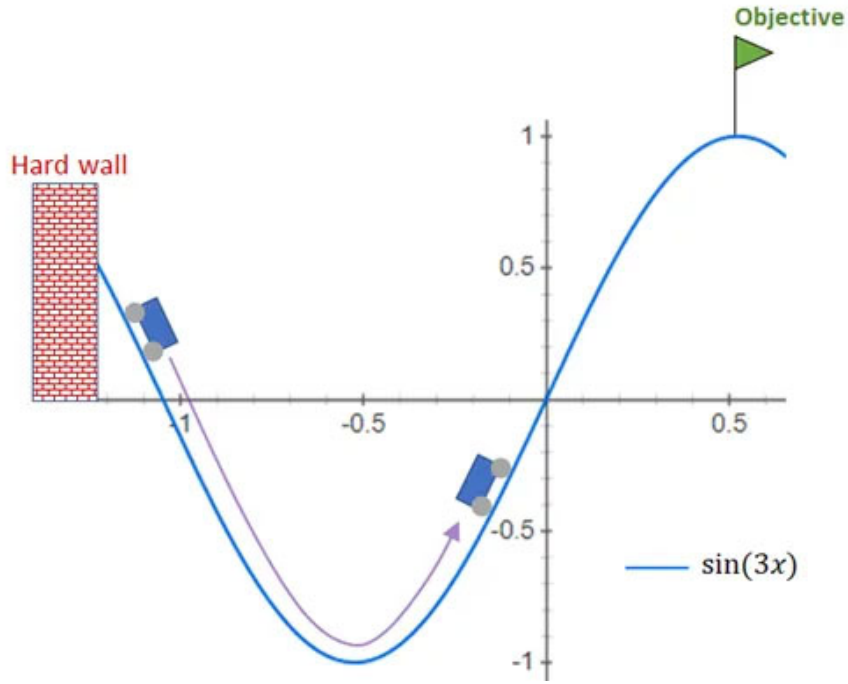
where force is the action clipped to the range [-1,1] and power is a constant 0.0015 and gravity = 0.0025

The collisions at either end are inelastic with the velocity set to 0 upon collision with the wall.

The position is clipped to the range [-1.2, 0.6] and velocity is clipped to the range [-0.07, 0.07].

Reward (Discrete)
The goal is to reach the flag placed on top of the right hill as quickly as possible, as such the agent is penalised with a reward of -1 for each timestep.
Reward (Continuous)
A negative reward of *-0.1 * action*^{2} is received at each timestep to penalise for taking actions of large magnitude.
If the mountain car reaches the goal then a positive reward of +100 is added to the negative reward for that timestep.

Starting State
The position of the car is assigned a uniform random value in *[-0.6 , -0.4]*.
The starting velocity of the car is always assigned to 0.



Episode End (Discrete)
The episode ends if either of the following happens:
1. Termination: The position of the car is greater than or equal to 0.5 (the goal position on top of the right hill)
2. Truncation: The length of the episode is 200.

Episode End (Continuous)
The episode ends if either of the following happens:
1. Termination: The position of the car is greater than or equal to 0.45 (the goal position on top of the right hill)
2. Truncation: The length of the episode is 999.

MountainCarEnv

Mountain Car has two parameters for gymnasium.make with render_mode and goal_velocity. On reset, the options parameter allows the user to change the bounds used to determine the new random state.

Arguments (Discrete)

```
env = gym.make("MountainCar-v0", render_mode="rgb_array", goal_velocity=0.1)
# default goal_velocity=0
env <TimeLimit<OrderEnforcing<PassiveEnvChecker<<MountainCar-v0>>>>>
env.reset(seed=123, options={"x_init": np.pi/2, "y_init": 0.5})
# default x_init=np.pi, y_init=1.0 (array([-0.46352962, 0. ], dtype=float32), {})
```

Arguments (Continuous)

```
env = gym.make("MountainCarContinuous-v0", render_mode="rgb_array", goal_velocity=0.1)
# default goal_velocity=0
env <TimeLimit<OrderEnforcing<PassiveEnvChecker<<MountainCarContinuous-v0>>>>>
env.reset(seed=123, options={"low": -0.7, "high": -0.5})
# default low=-0.6, high=-0.4 (array([-0.5635296, 0. ], dtype=float32), {})
```

MountainCar
Quick Summary

The **environment** is quite simple, and we only need to know two things about the car at every **state**, these are its **position** and its **velocity**. The problem is two dimensional, and so is its state space (the space with all possible positions and velocities).

The car only has 3 possible **actions** at every state, it can either accelerate forwards, accelerate backwards, or do nothing. There is also a configuration for continuous actions.

Every time the agent takes an action, the environment (the game) will return a new state (a position and velocity).

Scenarios to be analyzed

Possible Variation:
Minimum real “physical” time

	Minimum steps	Minimum fuel
Discrete	***MountainCar-v0*** **(1)**	*(Adapt)* Costs proportional to number of actions *[right,left]* taken **(3)**
Continuous	*(Adapt)* Costs linearly proportional to number of non-null actions taken **(4)**	***MountainCarContinuous-v0*** Costs proportional to the square of intensity of actions taken **(2)**

Appendix:

The essential physics concepts

Simple Harmonic Oscillator
(SHO)

$$x'' + \omega^2 x = 0$$

Damped Harmonic Oscillator
(DHO)

$$x'' + 2\beta x' + \omega_0^2 x = 0$$

Pendulum
(Pend)

$$\theta'' + \frac{g}{L}\sin\theta = 0$$

Anharmonic Oscillator
(AnHO)

$$x'' + \omega^2 x + \alpha x^3 = 0$$

Forced Harmonic Oscillator
(FHO)

$$x'' + 2\cancel{\beta x'} + \omega_0^2 x = F_0 \cos(\omega t)$$

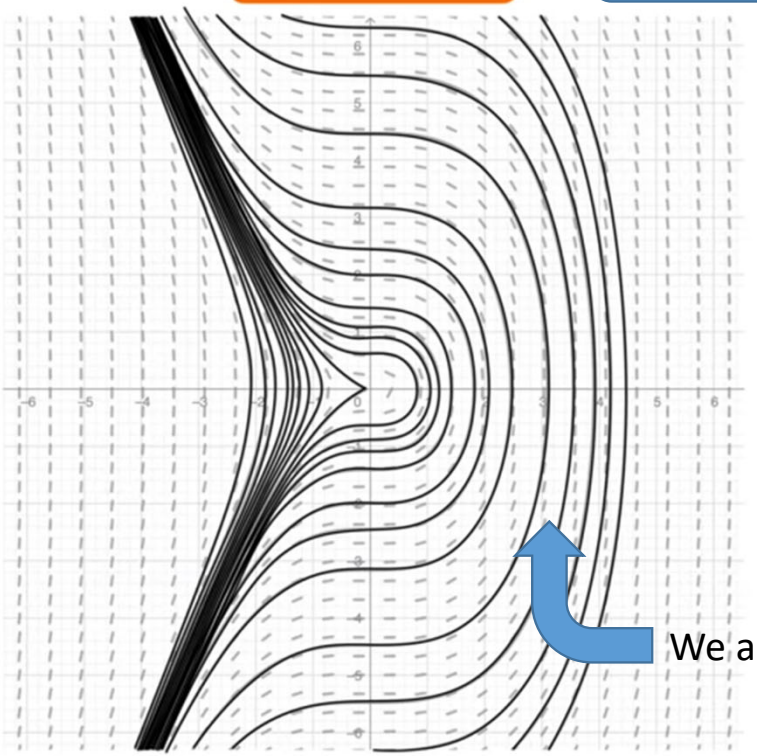
We are here…

$$x'' + x^n = 0$$

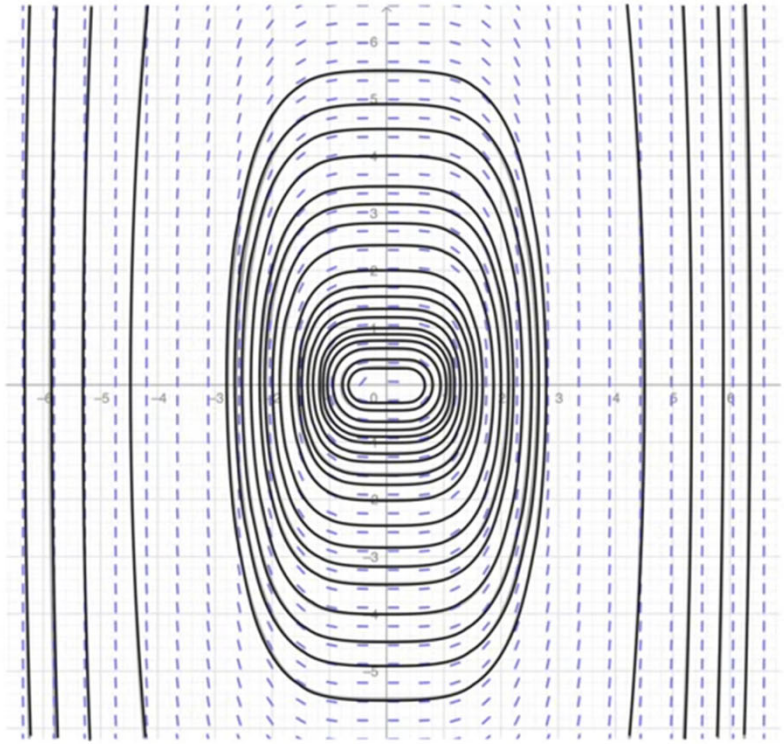
$n = 2$

Phase Portrait

$n = 3$



We are here…



Appendix:

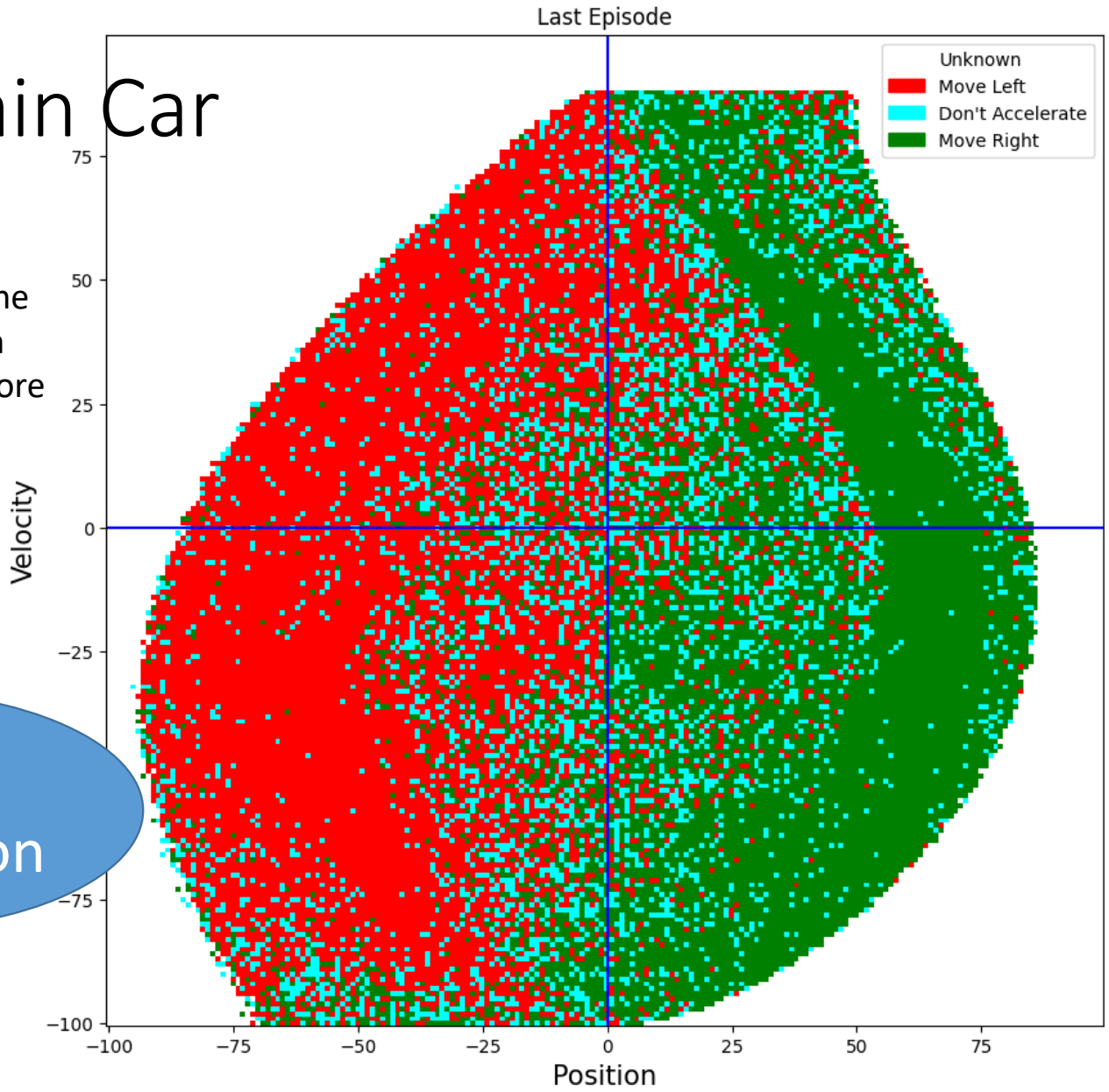
The essential physics concepts

Mountain Car QTable

We can approximate the value function through Q-learning, and therefore identify what the best action is at every state

Discretized space with N = 200.

Policy Visualization



The approximate value function is a little bit like the car's brain.
The car will simply do what ever is indicated by the value function.

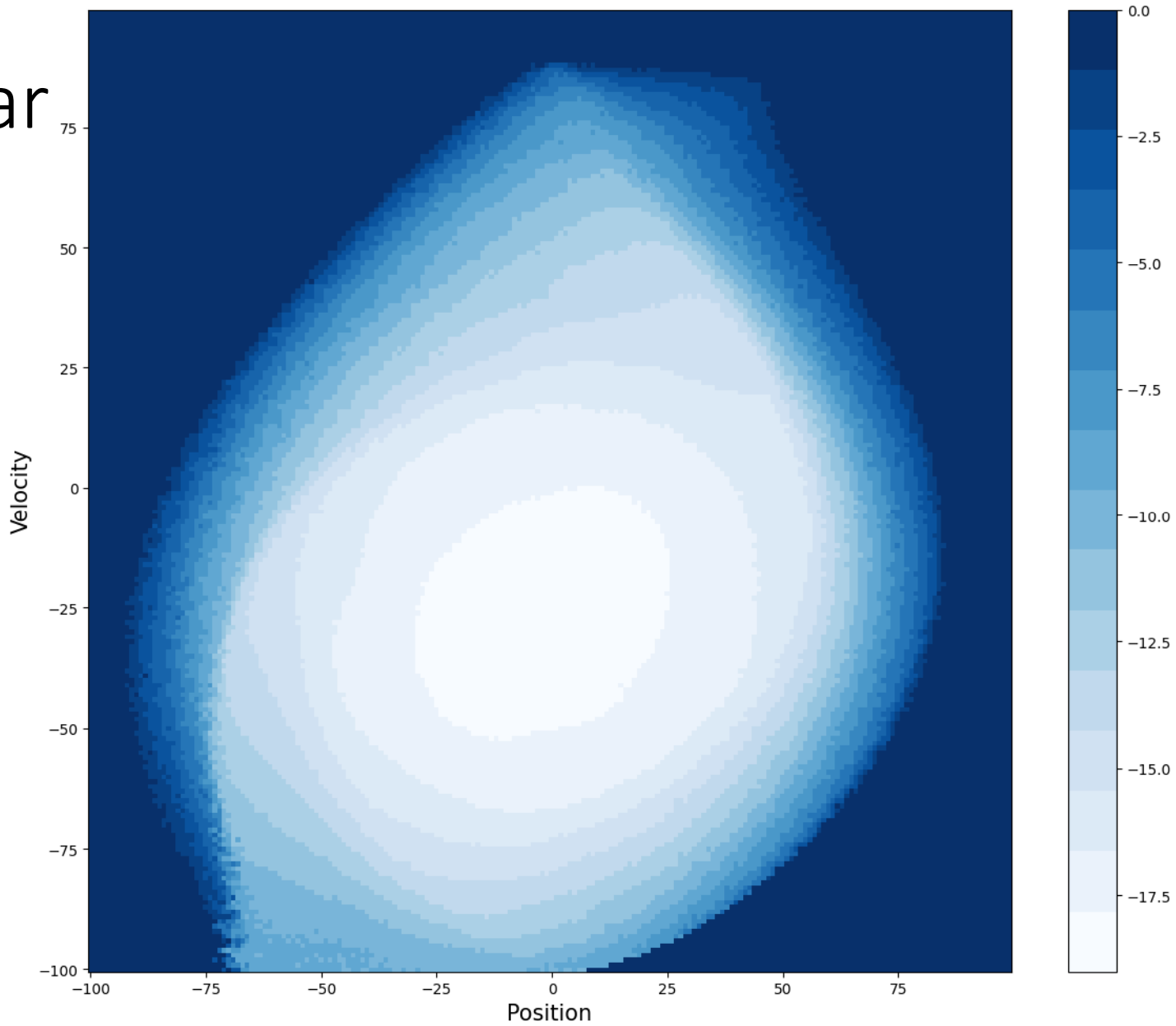
The value function is very complex, with a preference to move right when it is on the right side of the map, and a preference to move left on the left side of the map.

Learning of the approximate value function by the tabular method

Some observations

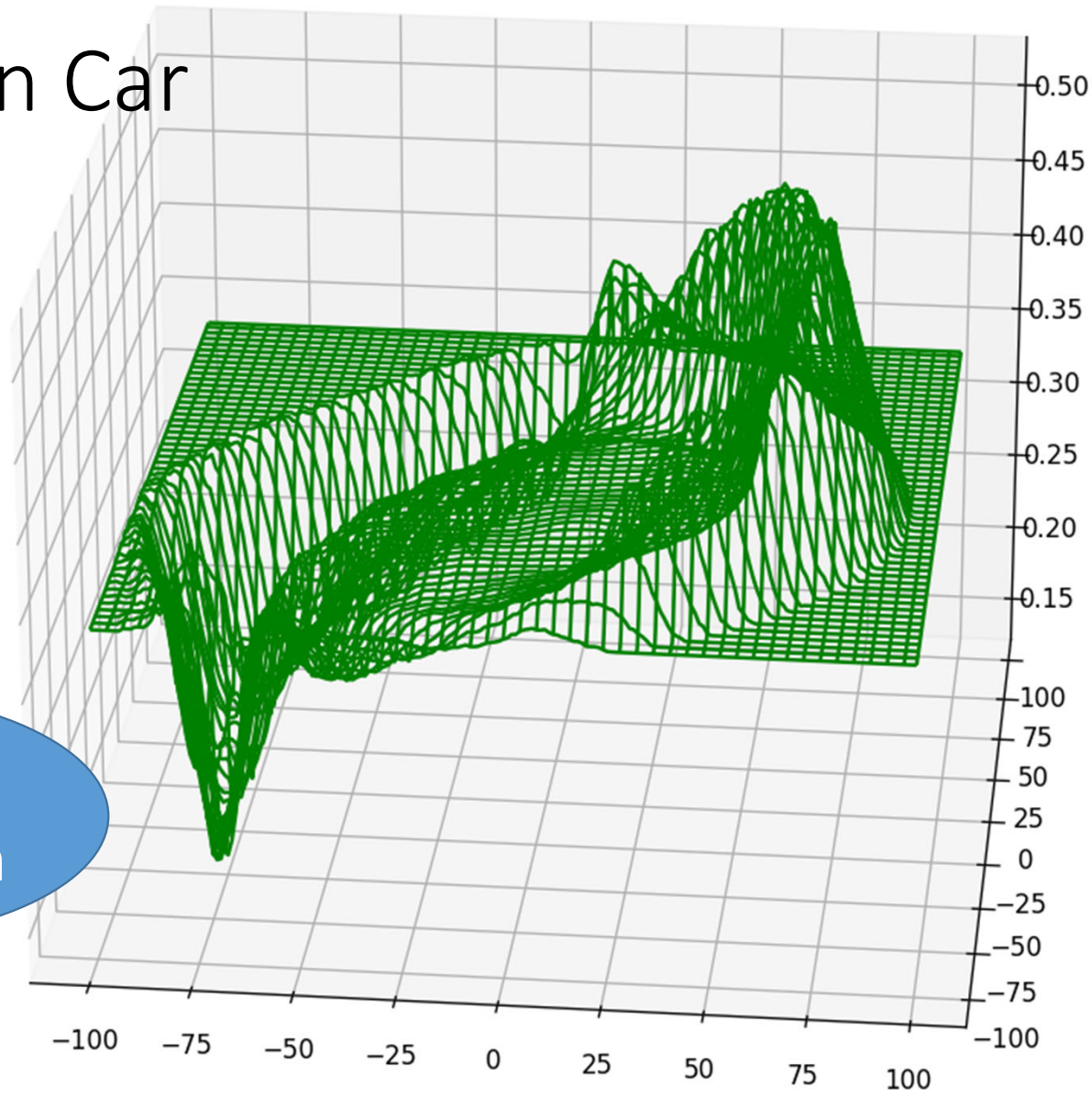
- The origin is slightly off center,
- The default action seems to be to accelerate right
- The model only updates the states it sees
- The model seems to grow its observed space with time
- The further you are from the origin the higher your energy (the higher the magnitude of your position or velocity).
- Aside from the chosen action, you can also plot the magnitude of the values. This indicates how many times the car has been in those states, and therefore gives us an idea of the confidence of the model, and what states it is most confident in.

Mountain Car QTable



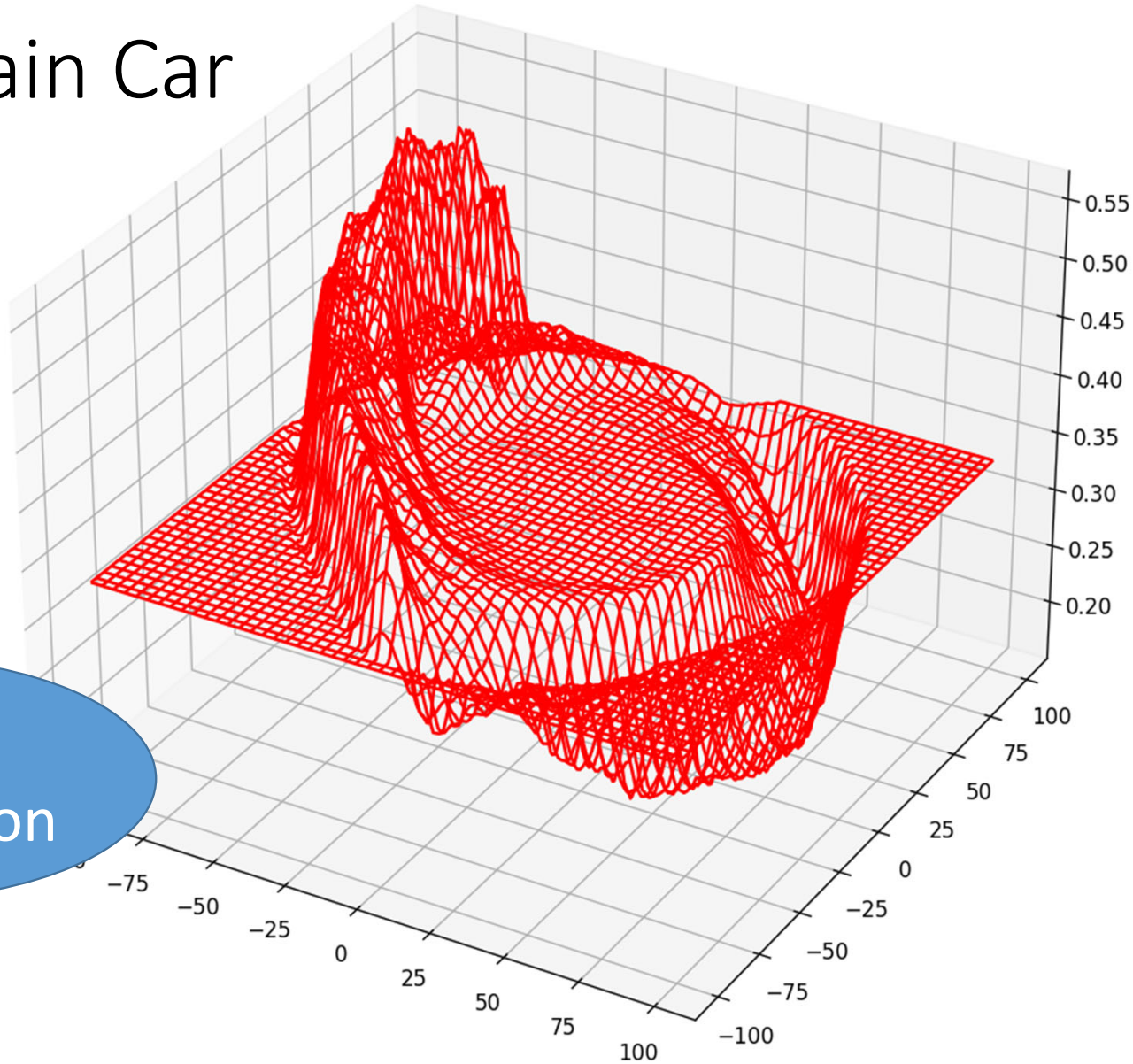
Frequency of visitation of states (as an estimator of confidence level)

Mountain Car QTable



Values Visualization

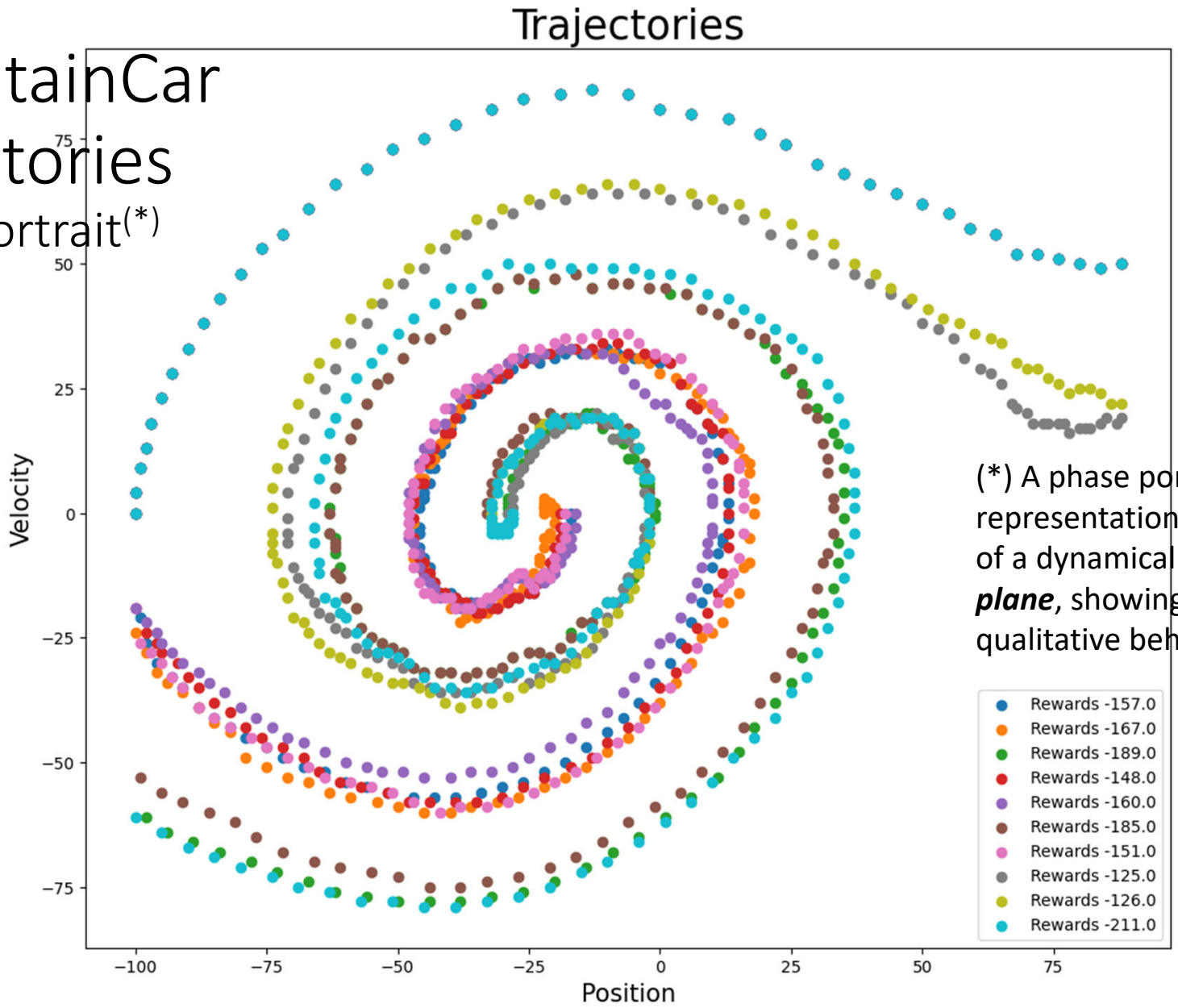
Mountain Car QTable



Values Visualization

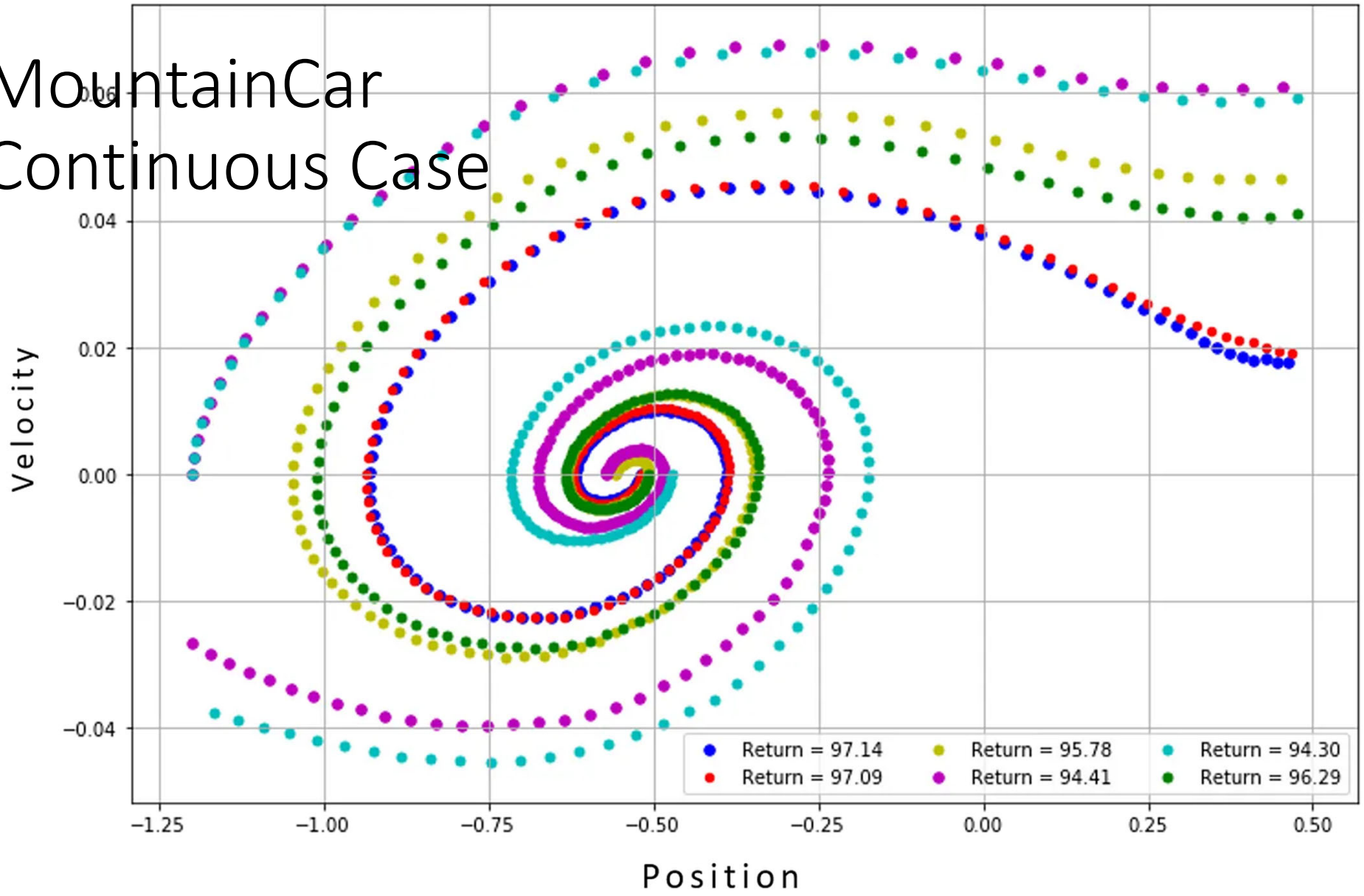
MountainCar Trajectories

Phase portrait(*)



(*) A phase portrait is a graphical representation of the trajectories of a dynamical system in the ***phase plane***, showing the system's qualitative behavior and stability.

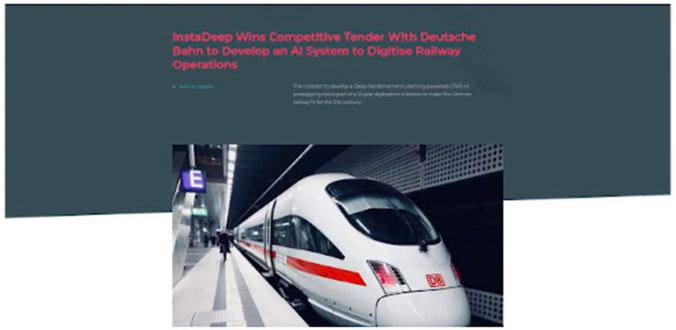
MountainCar Continuous Case



RL Applications List

Deep RL has been applied in many business problems

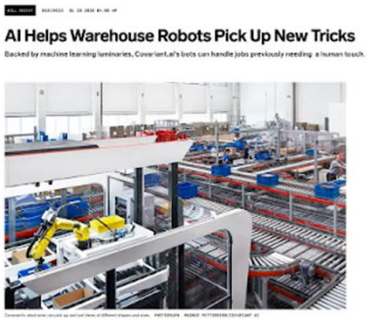
Energy usage reduction in data centers



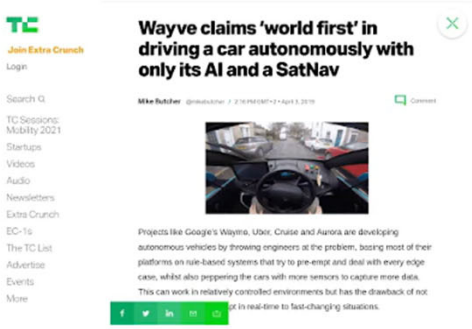
Managing fleet of trains

Trading stocks and portfolio management

Game bots



Warehouse robots

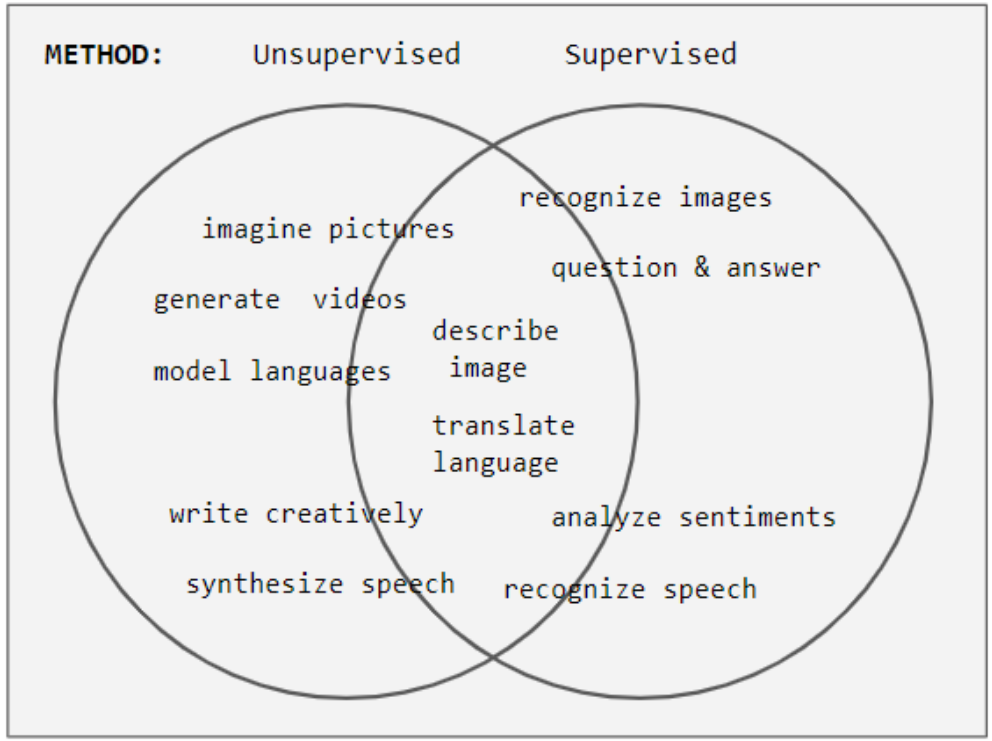


Self-driving cars

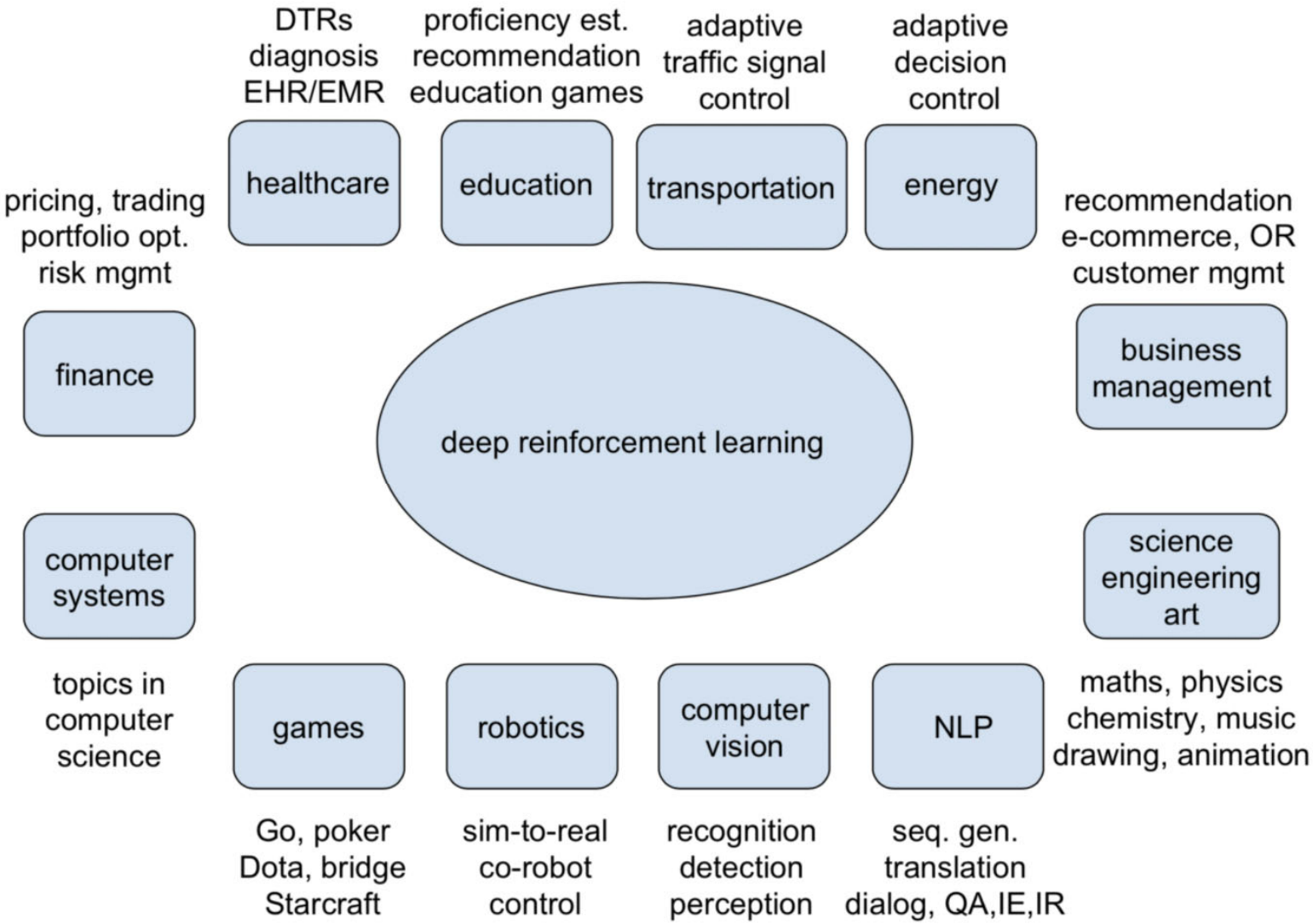
Applications of Reinforcement Learning
(orientative list for brainstorming)

- Autonomous cars
- Reduction in Energy Consumption
  - Datacenters cooling
- Traffic Control
  - Traffic light control
- Healthcare
- Image processing
- Robotics
- NLP
- Marketing
- Gaming
- Manufacturing
- Trading
- Recommenders
  - Online Recommendation
- Bid Optimization
- Marketing Strategies
- Industrial Machine Teaching
- Text Mining
- Simulation Modeling and Optimization
- Computerized Adaptive Testing
- Database systems
- Guidance, navigation and flight control
- Optimization of Computer Systems
- Market Optimization
- Optimizing Compilers
- Scientific advancement
- …

Some RL Examples



- Data Center Energy Usage Reduction
  - DeepMind AI Reduces Google Data Centre Cooling Bill by 40% — Google DeepMind
    - https://deepmind.google/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/
  - Google just gave control over data center cooling to an AI | MIT Technology Review
    - https://www.technologyreview.com/2018/08/17/140987/google-just-gave-control-over-data-center-cooling-to-an-ai/
- Trading in Financial Markets
  - JPMorgan develops robot to execute trades
    - https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c?syn-25a6b1a6=1
- Managing Fleets of Trains
  - InstaDeep Wins Competitive Tender With Deutsche Bahn to Develop an AI System to Digitise Railway Operations | InstaDeep - Decision-Making AI For The Enterprise
    - https://instadeep.com/2019/10/instadeep-wins-competitive-tender-with-deutsche-bahn-to-develop-an-ai-system-to-digitise-railway-operations/
- Stronger and More Realistic Game Bots
  - Gamemakers Inject AI to Develop More Lifelike Characters | WIRED
    - https://www.wired.com/story/game-makers-inject-ai-develop-more-lifelike-characters/
- Warehouse Robots
  - AI Helps Warehouse Robots Pick Up New Tricks | WIRED
  - https://www.wired.com/story/ai-helps-warehouse-bots-pick-new-skills/
- Self Driving Cars
  - Wayve claims 'world first' in driving a car autonomously with only its AI and a SatNav | TechCrunch
    - https://techcrunch.com/2019/04/03/wayve-claims-world-first-in-driving-a-car-autonomously-with-only-its-ai-and-a-satnav/



Yuxi Li, Deep Reinforcement Learning, arXiv, 2018

Additional References

(download from `ai.twilightparadox.com:2024`)

RL Applications (twilightparadox.com)
Raw data (JSON)

http://ai.twilightparadox.com:2024/static/_EXPERIMENTS/rl-applications-chart.html
http://ai.twilightparadox.com:2024/static/_EXPERIMENTS/rl-applications.json